Reconocimiento de símbolos musicales dibujados a mano

Una red neuronal DenseNet profunda que identifica 8 figuras trazadas con mouse o pantalla táctil

Anngie Salazar Álvarez
Código: 20162195F

Gabriel Arturo Rincón Miranda
Código: 20231285E

Erick Daniel Ortega Morán
Código: 20210209H

Facultad de Ciencias · Octubre 2026

EL PROBLEMA

Clasificar 8 símbolos musicales dibujados a mano

01 · Figura
Negra



02 · Figura
Blanca



03 · Alteración
Sostenido



04 · Alteración
Bemol



05 · Alteración
Becuadro



06 · Doble corchea
Doble negra



07 · Silencio
Silencio



08 · Clave
Clave sol



Problema de **clasificación multiclase en visión artificial**: el usuario traza libremente con mouse o pantalla táctil sobre una matriz monocroma de 64×64, requiriendo discriminar formas con trazos delgados, bucles y armaduras similares.

Del código base a nuestra metodología en 5 pasos

1

Recolectar

1 200 propios + 5 200 Rebelo.

2

Preparar

Centrado de masa y escala 64x64.

3

Entrenar

DenseNet (80k params) en PyTorch.

4

Evaluar

98.91% exactitud en prueba.

5

Desplegar

Demo web en Render con sesiones.

Evolución sobre el código base del curso (clasificador de letras U, N, I):

OBJETIVO Y CLASES

De 3 letras fijas a **8 figuras musicales** con trazo libre y discriminación morfológica fina.

DATOS Y CURACIÓN

De capturas aisladas a **6 400 imágenes balanceadas** (1 200 propias + 5 200 base Rebelo).

ARQUITECTURA Y DEMO

De trazo estático a **DenseNet profunda (80k params)** con inferencia en 14.6 ms y web multi-sesión.

PASO 1 · RECOLECCIÓN

Datos propios en lienzo y base pública Rebelo

LIENZO INTERACTIVO

Dibujamos con mouse y pantalla táctil en el lienzo interactivo de `recolector.py` para capturar la variabilidad de trazo real.

EQUILIBRIO DINÁMICO

El recolector solicita automáticamente la figura con menos muestras guardadas, garantizando paridad numérica exacta.

CONJUNTO HÍBRIDO

Integramos los 1 200 trazos interactivos con 5 200 partituras académicas de la base Rebelo (manuscritas e impresas).

150
dibujos propios por clase

1 200
dibujos propios en total

650
imágenes públicas por clase

6 400
total (800 / clase exactas)

PASO 2 · PREPARACIÓN

Estandarización morfológica de imágenes

1

Inversión binaria

Blanco sobre negro con umbral Otsu.

2

Recorte contorno

Ajuste milimétrico al trazo real.

3

Centrado de masa

Centro de gravedad morfológico.

4

Escala proporcional

Remuestreo bilineal a 64x64 px.

5

Normalización

Tensores continuos en [0.0, 1.0].

Partición estratégica de las 6 400 imágenes balanceadas:

Entrenamiento · 70%
4 480 imágenes (la red aprende las convoluciones)

10%
640 val

Prueba · 20%
1 280 imágenes aisladas

El 10% de validación monitorea la pérdida por época y previene sobreajuste. El conjunto de prueba de 1 280 imágenes se mantiene 100% ciego hasta la evaluación final.

Red convolucional densa (DenseNet)

La idea clave: conectividad por concatenación

A diferencia de redes que suman mapas (ResNet), en DenseNet cada capa concatena las salidas de todas las anteriores:

$$x_\ell = H_\ell([x_0, x_1, ..., x_{\ell-1}])$$

donde $H_\ell = \text{Conv}_{3\times3} \circ \text{ReLU} \circ \text{BatchNorm}$.

Ventaja: Reutiliza bordes y curvaturas primitivas aprendidas al inicio con poquísimos parámetros y sin desvanecimiento de gradiente.

80 520

parámetros entrenables totales

$k = 16$

tasa de crecimiento (filtros nuevos por capa)

Compresión en transiciones ($\theta = 0.5$):

Convolución 1×1 reduce canales al 50% y AvgPool 2×2 divide el tamaño a la mitad. Global Average Pooling (GAP) colapsa $92 \times 1 \times 1$.

Secuencia de capas y flujo en serpentina



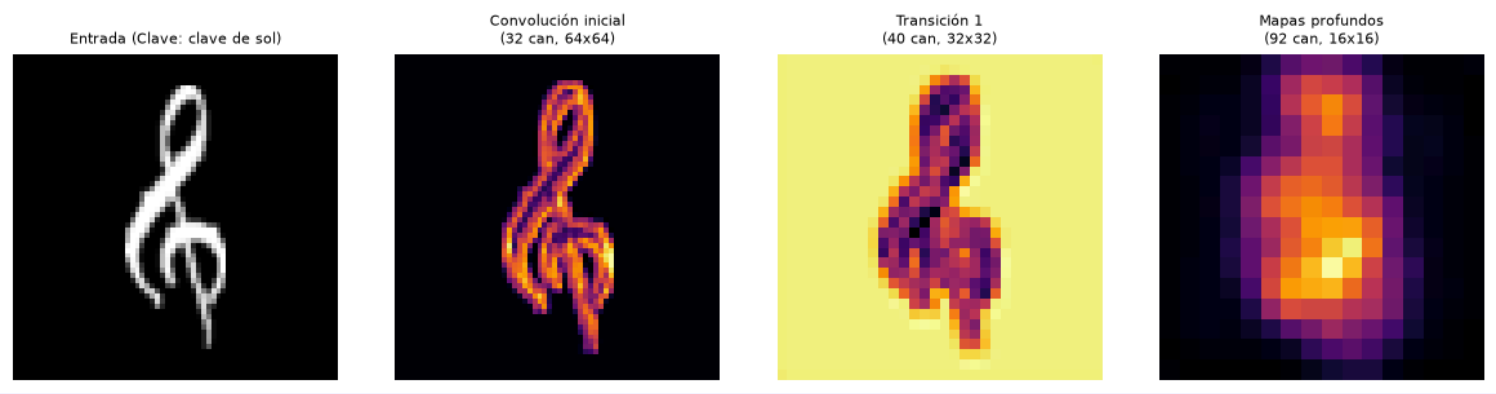
Etapas iniciales (propagación convolucional):

Entrada monocroma $1 \times 64 \times 64 \rightarrow$ Conv inicial (32 canales) $\rightarrow$ Bloque Denso 1 ($80 \times 64 \times 64$) $\rightarrow$ Transición 1 ($40 \times 32 \times 32$) $\rightarrow$ Bloque Denso 2 ($88 \times 32 \times 32$).

Etapas finales (abstracción, GAP y clasificación):

Transición 2 ($44 \times 16 \times 16$) $\rightarrow$ Bloque Denso 3 ($92 \times 16 \times 16$) $\rightarrow$ Normalización final $\rightarrow$ GAP $92 \times 1 \times 1 \rightarrow$ Clasificador lineal (8 probabilidades).

Evolución de la abstracción convolucional



Mapas de activación promedio extraídos en tiempo real durante la propagación hacia adelante

1. Convolución inicial

32 canales, 64×64

Filtros de primer orden: detección nítida de bordes, orientación y plicas.

2. Transición 1

40 canales, 32×32

Consolidación morfológica: condensación de uniones, grosor y bucles de la clave.

3. Mapas profundos

92 canales, 16×16

Abstracción semántica: activación holística del símbolo previa a la compresión global.

PASO 4 · ENTRENAMIENTO

15 épocas hasta estabilizarse en 99%

Optimizador

Adam, tasa 0.001

Función de pérdida

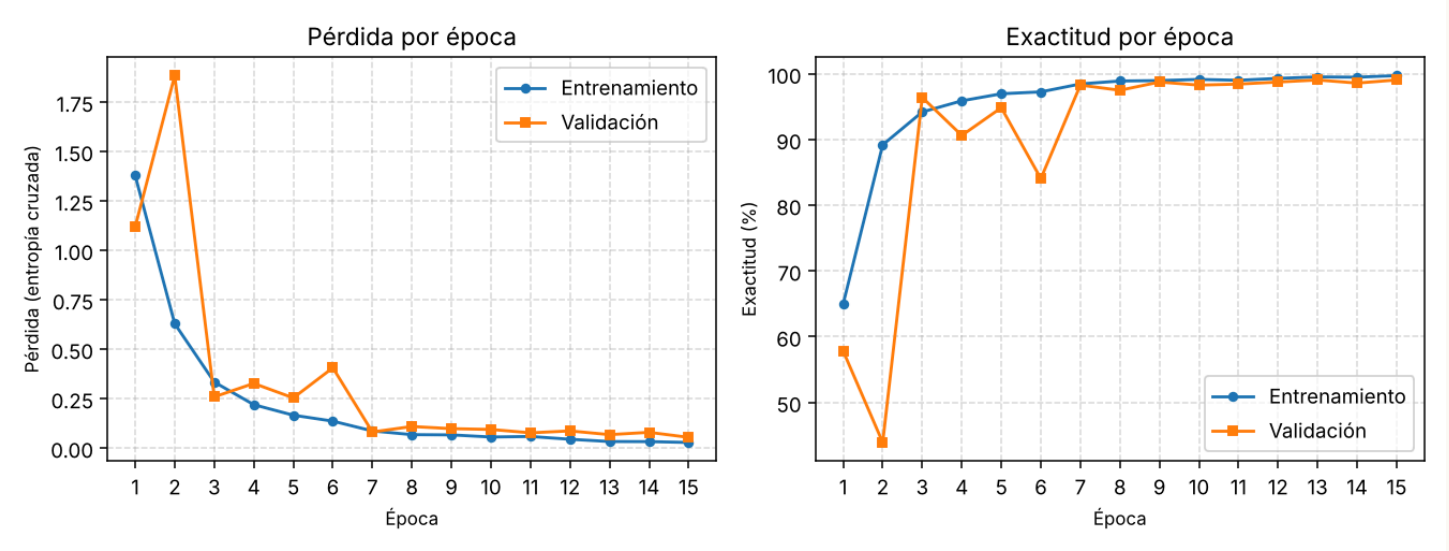
Entropía cruzada

Lote · épocas

32 · 15 épocas

Regularización

Decaimiento 10^{-4}



Curvas de convergencia en optimizador Adam (entrenamiento y validación)

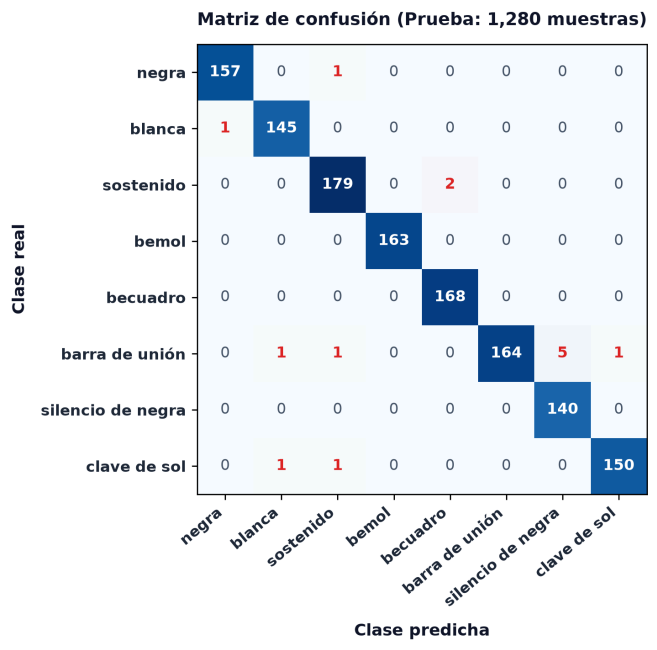
Al reducirse la pérdida a **0.038**, las curvas de entrenamiento y validación convergen de manera paralela sin sobreajuste. La exactitud de validación se estabiliza por encima del **98.5%** a partir de la época 8.

Acierta 98.9% de los símbolos que nunca vio

98.91%	99.69%	98.90%	14.6 ms
Exactitud Top-1	**Exactitud Top-2**	**F1 ponderado**	**Por predicción**
1 266 de 1 280 aciertos	La respuesta está en las 2 primeras opciones	Equilibrio armónico entre precisión y recall	En procesador CPU estándar (68 cuadros/segundo)

Medido rigurosamente sobre las 1 280 imágenes del conjunto de prueba independiente, separadas desde el inicio y nunca vistas durante el entrenamiento.

Matriz de confusión (1 280 muestras de prueba)



1 266 / 1 280

Aciertos directos en prueba (98.91%)

Diagnóstico morfológico de casos clave:

- **Doble negra (corcheas con barra):** Antes solo se tenían líneas solas. Al incorporar dibujos manuales con plicas, se reconoció la figura completa (95.3% recall, 5 confusiones por diagonales similares al silencio).
- **Silencio de negra:** La alta varianza caligráfica en los datos originales hacía muy difícil su reconocimiento (83.6% previo). La base manual adicional estabilizó el símbolo alcanzando **100.0% de aciertos**.
- **Cero falsos positivos** entre bemol y becuadro.

Precisión, recall y F1-score por figura

Figura / Símbolo	Precisión	Recall	F1-Score	Muestras prueba
♩ Negra	99.4%	99.4%	99.4%	158
𝅗𝅥 Blanca	98.6%	99.3%	99.0%	146
♯ Sostenido	98.4%	98.9%	98.6%	181
♭ Bemol	**100.0%**	**100.0%**	**100.0%**	**163**
♮ Becuadro	98.8%	100.0%	99.4%	168
♫ Doble negra (corcheas unidas)	**100.0%**	**95.3%**	**97.6%**	**172**
𝄽 Silencio de negra	96.6%	100.0%	98.3%	140
𝄞 Clave de sol	99.3%	98.7%	99.0%	152

Destacado: El **bemol** y el **silencio de negra** alcanzaron un recall perfecto del **100.0%** en prueba ciega (163/163 y 140/140 aciertos directos).

Los dibujos propios bajaron los errores de 49 a 14

El valor de los datos interactivos:

- **Doble negra (beams):** De reconocer únicamente líneas de unión aisladas a clasificar la figura completa con plicas trazada por el usuario.
- **Silencio de negra:** La alta varianza original se resolvió con trazos reales, subiendo el recall del 83.6% al **100.0%**.
- **Erradicación de confusiones:** Antes 19 bemoles se confundían con becuadro; ahora acierta **163 de 163**.

Métrica en prueba	Solo base pública	Con dibujos propios
Exactitud Top-1	96.17%	**98.91%**
Exactitud Top-2	98.83%	**99.69%**
F1 ponderado	96.19%	**98.90%**
Errores en prueba	49 errores	**Solo 14 errores**
Peor recall de clase	83.6% (silencio)	**95.3% (doble negra)**

Evaluación sobre el conjunto de prueba independiente de 1 280 imágenes.

Módulo de inferencia y preprocesamiento

`MusicalSymbolPredictor.predict`

- **Firma:** `predict(img_64x64, top_k=4)`
- **Entrada:** Matriz NumPy $(64, 64)$ en uint8 $[0, 255]$.
- **Salida:** Diccionario estructurado con:
  - `label`: Clase ganadora en texto.
  - `confidence`: Probabilidad en porcentaje.
  - `top_k`: Lista ordenada de las k mejores clases.
  - `probabilities`: Vector completo Softmax de 8 clases.
- **Optimización:** Inferencia con `torch.no_grad()` para evitar consumo de memoria en gradientes.

Familia `preprocess_*`

- **Ingesta polimórfica:**
  - `preprocess_file(path)` para imágenes en disco.
  - `preprocess_pil(image)` para objetos PIL en memoria.
  - `preprocess_base64(data_uri)` para el lienzo web.
- **Canalización interna:**
  1. Conversión monocroma e inversión binaria Otsu.
  2. Bounding box automático alrededor de los píxeles de trazo.
  3. Centrado morfológico por centro de masa.
  4. Escalamiento proporcional a matriz estándar de 64×64.

Estructura del modelo y generadores

`DenseNetMusical.forward`

- **Entrada:** Tensor PyTorch $(B, 1, 64, 64)$ en $[0.0, 1.0]$.
- **Flujo:** Conv inicial $\rightarrow$ DenseBlock 1 $\rightarrow$ Trans 1 $\rightarrow$ DenseBlock 2 $\rightarrow$ Trans 2 $\rightarrow$ DenseBlock 3 $\rightarrow$ GAP.
- **Salida:** Tensor de logits lineales $(B, 8)$ previo a Softmax.

`get_intermediate_activations`

Extrae tensores de 3 etapas clave para visualizarlos en tiempo real en la interfaz web interactiva.

`create_dataloaders`

- **Firma:** `create_dataloaders(batch_size=32)`
- **Partición:**
  - 70% Entrenamiento (4 480 muestras).
  - 10% Validación (640 muestras).
  - 20% Prueba independiente (1 280 muestras).
- **Determinismo:** Semilla fija (42) y barajado reproducible con generadores pseudoaleatorios de PyTorch.
- **Estructura:** Carga desde arrays binarios serializados `dataset/X.npy` e `y.npy`.

Demostración en vivo en la nube

https://clasificador-de-simbolos-musicales.onrender.com

1

Acceso público

Desplegado en producción en Render con Python, Flask y Gunicorn.

2

Dibujar símbolo

Lienzo táctil e interactivo en tiempo real desde celular o PC.

3

Ver predicción

Top 4 probabilidades con porcentaje y mapas de activación en vivo.

Aislamiento concurrente: Cada pestaña o navegador maneja su propia sesión (`SessionSpace`) independiente iniciando con 0 aciertos. Nota: Tras 15 minutos de inactividad la instancia gratuita suspende (**cold start** de 30-50 s).

Balance del proyecto

- **Adaptamos el código del curso** a un problema nuevo de clasificación de 8 figuras musicales en visión artificial.
- **La recolección de datos propios** fue el factor determinante: redujo los errores en prueba de 49 a solo 14.
- **Una DenseNet compacta (80 mil parámetros)** supera a modelos secuenciales pesados alcanzando 98.91% de exactitud.
- **Respuesta en tiempo real (14.6 ms en CPU):** viabilidad total para una web interactiva sin GPU dedicada.

¿Preguntas?

Muchas gracias

Universidad Nacional de Ingeniería · CC431A